SOFTWARE REQUIREMENTS SPECIFICATION

CSE 816: Software Production Engineering

Final Project — DevOps-Driven Fraud Detection System

Document Version:	1.0
Project Domain:	FinTech / Fraud Detection (AIOps)
Prepared By:	Kautilya
Date:	April 2026
Course Instructor:	[Instructor Name]
Total Marks:	25 (Target: Full marks)

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the complete functional and non-functional requirements for a DevOps-driven Fraud Detection System developed as the final project for CSE 816: Software Production Engineering. The system integrates a machine learning inference service with a fully automated DevOps pipeline spanning version control, continuous integration and delivery, containerisation, configuration management, orchestration, secret management, and observability.

This document is intended to serve as the definitive reference for implementation, evaluation, and demonstration. Every requirement stated herein maps directly to the project evaluation rubric defined in the course guidelines and to a specific component of the implemented system.

1.2 Scope

The project, titled FraudGuard DevOps Platform, is a domain-specific (FinTech) implementation of a complete DevOps lifecycle automation framework. The scope encompasses:

- A Python FastAPI application that serves a trained scikit-learn Random Forest fraud detection model trained on the IEEE-CIS Fraud Detection dataset from Kaggle.
- A fully automated CI/CD pipeline implemented in Jenkins, triggered by GitHub webhook events and capable of building, testing, packaging, and deploying the application without manual intervention.
- Docker-based containerisation with a single production image published to Docker Hub on every successful pipeline run.
- Ansible-based infrastructure provisioning of the Minikube virtual machine using modular roles.
- Kubernetes orchestration with Horizontal Pod Autoscaler (HPA) to demonstrate dynamic scaling.
- ELK Stack (Elasticsearch, Logstash, Kibana) deployed inside the Kubernetes cluster, consuming structured application logs via a Filebeat sidecar.
- HashiCorp Vault for centralised secret management, accessed by Jenkins at runtime via AppRole authentication.

The following are explicitly out of scope for this project: a production cloud deployment, multi-tenant user authentication, a real-time financial data feed, and a frontend web UI beyond the FastAPI auto-generated documentation interface.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification — this document
DevOps	Development and Operations — a set of practices combining software development and IT operations
SDLC	Software Development Life Cycle

Term / Acronym	Definition
CI/CD	Continuous Integration / Continuous Delivery — automated build, test, and deployment pipeline
ML	Machine Learning
RF	Random Forest — ensemble machine learning algorithm used for fraud classification
IEEE-CIS	IEEE Computational Intelligence Society Fraud Detection Dataset (Kaggle, 2019)
FastAPI	A modern, high-performance Python web framework for building APIs
K8s	Kubernetes — container orchestration system
HPA	Horizontal Pod Autoscaler — K8s mechanism to scale pods based on CPU/memory metrics
ELK	Elasticsearch, Logstash, and Kibana — a log ingestion and visualisation stack
VM	Virtual Machine
DXA	Document Extensible Attribute unit used in DOCX formatting (1440 DXA = 1 inch)
AppRole	Vault authentication method where applications log in with a Role ID and Secret ID
ngrok	A reverse-proxy tunnelling tool that exposes localhost to the public internet
IaC	Infrastructure as Code — managing infrastructure through machine-readable configuration files
PKL	Python Pickle format used to serialise trained scikit-learn model objects
FR	Functional Requirement
NFR	Non-Functional Requirement
emptyDir	A Kubernetes ephemeral volume shared between containers in the same pod

1.4 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- Vancheri, A. (2019). IEEE-CIS Fraud Detection Dataset. Kaggle. https://www.kaggle.com/c/ieee-fraud-detection
- HashiCorp Vault Documentation v1.16: https://developer.hashicorp.com/vault/docs
- Jenkins User Handbook: https://www.jenkins.io/doc/book/
- Kubernetes Documentation v1.29: https://kubernetes.io/docs/
- Elastic Stack Documentation 8.x: https://www.elastic.co/guide/
- Ansible Documentation v9: https://docs.ansible.com/
- CSE 816 Final Project Guidelines and Evaluation Criteria (Provided PDF)

1.5 Document Overview

The remainder of this document is organised as follows: Section 2 provides an overall description of the system. Sections 3 through 9 detail functional requirements for each subsystem. Section 10 covers non-functional requirements. Section 11 maps requirements to evaluation marks. Section 12 provides build-order guidance and risk mitigation strategies.

2. Overall System Description

2.1 Product Perspective

FraudGuard DevOps Platform is a self-contained academic demonstration system that simulates an enterprise-grade MLOps/FinTech deployment pipeline. The system is not a component of a larger existing product; it is purpose-built to demonstrate full SDLC automation using the tools mandated by CSE 816.

The system operates on a single physical host machine running two logical environments: the Host Environment (Jenkins and HashiCorp Vault) and a Guest Virtual Machine (VirtualBox/VMware) running Minikube, which hosts the Kubernetes cluster, the application workloads, and the ELK stack. Ansible playbooks execute from the host and provision the guest VM over SSH.

2.2 Product Functions (Summary)

At the highest level, the system performs the following functions:

1. Accept a financial transaction payload via HTTP POST to the /predict endpoint and return a fraud classification (0 = legitimate, 1 = fraudulent) along with a confidence score.
2. Detect every Git push to the main branch of the GitHub repository and automatically trigger a Jenkins pipeline that rebuilds, retests, repackages, and redeploys the application without any manual intervention.
3. Publish the newly built Docker image to Docker Hub, tagged with the Git commit SHA and the latest tag, using credentials fetched securely from HashiCorp Vault.
4. Provision and configure the Minikube VM idempotently via Ansible playbooks using modular roles, ensuring the VM is always in the correct state before deployment.
5. Deploy the updated Docker image to the Kubernetes cluster using a rolling update strategy, ensuring zero downtime during deployments.
6. Autoscale the number of application pods between a minimum of 2 and a maximum of 6 based on CPU utilisation exceeding 70%, as managed by the Kubernetes HPA.
7. Ship every application log entry (including prediction request details, confidence scores, and fraud flags) to Logstash via a Filebeat sidecar, and make them queryable and visualisable in Kibana.
8. Store all sensitive credentials (Docker Hub password, GitHub PAT, Minikube kubeconfig) in HashiCorp Vault and retrieve them at Jenkins pipeline runtime via AppRole authentication.

2.3 User Classes and Characteristics

User Class	Description	Interaction Mode
Developer	The student author who writes application code, pushes to GitHub, and monitors pipeline execution	Git CLI, Jenkins UI, FastAPI docs (/docs)
DevOps Engineer	Same as developer in this project; manages Ansible playbooks, K8s	SSH, kubectl CLI, Ansible CLI, Vault UI

User Class	Description	Interaction Mode
	manifests, Vault policies	
Evaluator	Course instructor who assesses the live demonstration against the rubric	Browser (Kibana, Jenkins, FastAPI /docs), Terminal
API Consumer	Any HTTP client (Postman, Newman, curl) that sends transaction payloads to the /predict endpoint	HTTP POST requests

2.4 Operating Environment

2.4.1 Host Machine

- OS: Windows 10/11 or Ubuntu 22.04 LTS
- RAM: 16 GB (minimum 6 GB available for VMs after host OS overhead)
- CPU: x86_64 architecture, hardware virtualisation enabled (Intel VT-x or AMD-V) in BIOS
- Disk: At least 80 GB free for VM images, Docker layers, and build artefacts
- Software: VirtualBox 7.x or VMware Workstation 17, Jenkins LTS, HashiCorp Vault 1.16, ngrok Agent, Git 2.x, Docker CLI

2.4.2 Virtual Machine (VM1 — Minikube Host)

- OS: Ubuntu 22.04 LTS (minimal server install)
- vCPU: 4 cores allocated
- RAM: 10 GB allocated
- Disk: 50 GB dynamically allocated virtual disk
- Network: NAT with port forwarding for SSH (2222 -> 22) and Kibana NodePort (30601 -> 30601) and FastAPI NodePort (30080 -> 30080)
- Software (provisioned by Ansible): Docker Engine 24.x, kubectl 1.29, Minikube 1.32, metrics-server addon enabled

2.4.3 External Services

- GitHub (github.com): Hosts the application source repository; sends webhook events to Jenkins via ngrok tunnel
- Docker Hub (hub.docker.com): Registry for published Docker images
- ngrok (ngrok.io): Provides a stable public HTTPS URL tunnelling to Jenkins on localhost:8080

2.5 Design and Implementation Constraints

- The system must operate entirely on local machines with no mandatory cloud provider dependency. All cloud references are optional alternatives.

- The Docker image must be a single image (no separate model-server sidecar). The scikit-learn model file (model.pkl) must be COPY'd into the image at build time.
- All secrets (Docker Hub credentials, GitHub Personal Access Token, Minikube kubeconfig file) must be stored exclusively in HashiCorp Vault. Hardcoding credentials in Jenkinsfiles, playbooks, or source code is strictly prohibited.
- The Kubernetes HPA must use the CPU utilisation metric (not custom metrics) via the metrics-server addon to avoid requiring a Prometheus adapter, which would exceed timeline constraints for a solo project.
- The ELK stack must be deployed as Kubernetes Deployments inside the Minikube cluster — not as standalone Docker containers on the host — to demonstrate K8s-native deployment of infrastructure components.
- Log shipping from the application pod to Logstash must use the Filebeat sidecar pattern with a shared emptyDir volume, not a DaemonSet (Minikube single-node limitation).
- The Jenkins pipeline must have at least the following seven distinct stages: Checkout, Unit Test, Docker Build, Docker Push, Ansible Provision, Kubernetes Deploy, and Newman Smoke Test.

2.6 Assumptions and Dependencies

- The developer has a GitHub account and a Docker Hub account with free-tier access.
- The ngrok agent is authenticated with a valid ngrok account. The static domain feature (free tier) is used to avoid session expiry breaking the webhook URL.
- The IEEE-CIS Fraud Detection dataset has been pre-downloaded from Kaggle and the Random Forest model has been trained and serialised to model.pkl before the first Docker build.
- The VirtualBox/VMware VM has been created with an Ubuntu 22.04 ISO and is accessible via SSH from the host before Ansible is run.
- Python 3.11 is used as the runtime both for the application and for the model training script.
- Vault is started in dev mode for this academic project (vault server -dev). Production Vault mode is out of scope.
- Jenkins runs as a local service on the host machine, accessible at http://localhost:8080.

3. Application Layer Requirements

This section specifies requirements for the FastAPI fraud detection service — the application component that all DevOps tooling is built around.

3.1 Machine Learning Model

3.1.1 Dataset

- FR-001: The system shall use the IEEE-CIS Fraud Detection dataset (Kaggle 2019) comprising 590,540 transaction records with 434 features across transaction and identity tables.
- FR-002: Feature engineering shall produce a minimum of 30 numeric and categorical features after merging TransactionID across both tables, handling missing values with median/mode imputation, and encoding categorical columns using LabelEncoder.
- FR-003: The training script (train.py) shall split the dataset with a 80:20 train-test ratio using a fixed random_state=42 for reproducibility.

3.1.2 Model Training

- FR-004: The model shall be a scikit-learn RandomForestClassifier with the following hyperparameters: n_estimators=100, max_depth=20, min_samples_split=5, class_weight='balanced' (to handle the class imbalance inherent in fraud datasets), random_state=42.
- FR-005: The training script shall output a classification report including precision, recall, F1-score, and ROC-AUC score to stdout upon completion.
- FR-006: The trained model shall achieve a minimum ROC-AUC score of 0.85 on the test split before being accepted and serialised. If this threshold is not met, the training script shall exit with a non-zero return code.
- FR-007: The trained model shall be serialised to model.pkl using joblib.dump() (not pickle.dump) to ensure compatibility across Python environments.

3.1.3 Model Artefact Packaging

- FR-008: The model.pkl file shall be tracked in the Git repository via Git LFS (Large File Storage) to avoid bloating the repository history with binary files.
- FR-009: The Dockerfile shall COPY model.pkl into the image at /app/model.pkl at build time. No runtime volume mount shall be required for the model file.

3.2 FastAPI Application

3.2.1 Endpoints

Endpoint	Method	Auth Required	Description
/predict	POST	No	Accepts a JSON payload of transaction features and returns fraud prediction and confidence score

Endpoint	Method	Auth Required	Description
/health	GET	No	Returns HTTP 200 with {status: healthy, model_loaded: true} if model is loaded; 503 if model failed to load
/metrics	GET	No	Exposes Prometheus-format metrics (request count, prediction latency) via prometheus-fastapi-instrumentator library
/docs	GET	No	FastAPI auto-generated Swagger UI for interactive API exploration during evaluation demo

3.2.2 /predict Request Schema

- FR-010: The POST /predict endpoint shall accept a JSON body conforming to a Pydantic model named TransactionRequest. The model shall include fields matching the top 30 features selected during training (e.g., TransactionAmt, ProductCD, card1–card6, addr1, addr2, dist1, P_emaildomain, R_emaildomain, and selected V-features).
- FR-011: The endpoint shall validate all incoming fields using Pydantic's built-in validation. Missing required fields shall return HTTP 422 Unprocessable Entity with a detailed error body listing the missing fields.
- FR-012: Numeric fields shall accept both integer and float input. Categorical fields shall accept string values.

3.2.3 /predict Response Schema

- FR-013: A successful prediction response (HTTP 200) shall return a JSON object with the following fields: transaction_id (string, echoed from request or UUID-generated if not provided), prediction (integer, 0 or 1), label (string, 'legitimate' or 'fraudulent'), confidence_score (float, 0.0–1.0, representing the RF predict_proba output for the predicted class), and timestamp (ISO 8601 string, UTC).
- FR-014: If the model is not loaded, /predict shall return HTTP 503 Service Unavailable with an error message.
- FR-015: If prediction throws an unexpected exception, /predict shall return HTTP 500 with the exception type and message (not a stack trace) and shall log the full exception with traceback to the application log file at ERROR level.

3.2.4 Structured Logging

- FR-016: The application shall use Python's standard logging module configured with a JSON formatter (python-json-logger library). Every log record shall include: timestamp (ISO 8601 UTC), level, message, module, and for prediction requests: transaction_id, prediction, confidence_score, processing_time_ms, and client_ip.
- FR-017: Log output shall be written to both stdout (for container runtime capture) and to a file at /var/log/app/fraud.log inside the container. The /var/log/app directory shall be created by the Dockerfile.

- FR-018: The application shall log at INFO level for every successful prediction, WARN level for transactions with confidence_score below 0.6 (low-confidence predictions), and ERROR level for any exception during prediction.

3.3 Application Dependencies and Dockerfile

3.3.1 Python Dependencies (requirements.txt)

Package	Version	Purpose
fastapi	>=0.110.0	Web framework for the prediction API
uvicorn[standard]	>=0.29.0	ASGI server to run FastAPI
scikit-learn	==1.4.2	ML library (pinned for model pkl compatibility)
pandas	>=2.2.0	Data manipulation for feature preprocessing
numpy	>=1.26.0	Numerical operations
joblib	>=1.3.0	Model serialisation and deserialisation
python-json-logger	>=2.0.7	JSON-format log output
prometheus-fastapi-instrumentator	>=6.1.0	Exposes /metrics endpoint for Prometheus
pydantic	>=2.6.0	Request and response schema validation

3.3.2 Dockerfile Specification

- FR-019: The Dockerfile shall use python:3.11-slim as the base image to minimise final image size.
- FR-020: The Dockerfile shall use a multi-stage build: a builder stage (installs dependencies from requirements.txt into a virtual environment) and a final stage (copies only the venv and application code, discarding build tools). This shall produce an image under 600 MB.
- FR-021: The WORKDIR shall be set to /app. The application shall run as a non-root user (useradd appuser && chown appuser:appuser /app) for security hardening.
- FR-022: The Dockerfile shall create the directory /var/log/app and set ownership to appuser before switching to the non-root user.
- FR-023: The container shall EXPOSE port 8000. The CMD shall be: uvicorn main:app --host 0.0.0.0 --port 8000.
- FR-024: The Docker image shall be tagged with two tags on every pipeline build: [dockerhub_username]/fraudguard:latest and [dockerhub_username]/fraudguard:[git_commit_sha_8chars].

4. Version Control Requirements

4.1 Repository Structure

- FR-025: The project shall be hosted in a public or private GitHub repository named fraudguard-devops.
- FR-026: The repository shall follow the directory structure below:

```
fraudguard-devops/   app/                  # FastAPI application source     main.py
# Application entry point     model.py            # Model loading and prediction logic
schemas.py          # Pydantic request/response models     logging_config.py   # JSON
logger setup     train.py            # Offline model training script     model.pkl
# Trained model (tracked via Git LFS)     requirements.txt    # Python dependencies
tests/               # Test suite     test_predict.py     # pytest unit tests
postman/            # Newman collection       FraudGuard.postman_collection.json
docker/                   Dockerfile          # Multi-stage build     docker-
compose.yml  # Local dev compose   ansible/     site.yml            # Master playbook
inventory/hosts.ini # VM SSH details     roles/       common/tasks/main.yml
docker/tasks/main.yml       kubernetes/tasks/main.yml   k8s/     deployment.yaml     #
App Deployment + HPA     service.yaml        # NodePort Service     elk/
# ELK K8s manifests       elasticsearch.yaml       logstash.yaml       kibana.yaml
Jenkinsfile           # Declarative pipeline   README.md
```

4.2 Branching and Commit Strategy

- FR-027: The repository shall have a main branch as the protected production branch. All direct pushes to main shall trigger the Jenkins CI/CD pipeline.
- FR-028: Development work shall be performed on feature branches following the naming convention feature/<short-description> (e.g., feature/add-hpa). Feature branches are merged to main via pull requests.
- FR-029: Commit messages shall follow the Conventional Commits specification: type(scope): description (e.g., feat(api): add confidence_score to predict response).
- FR-030: The repository shall include a .gitignore file that excludes: __pycache__, .env, *.pyc, venv/, .DS_Store, and any .pkl file not tracked by Git LFS.
- FR-031: Git LFS shall be configured with a .gitattributes file that tracks *.pkl files: *.pkl filter=lfs diff=lfs merge=lfs -text.

4.3 GitHub Webhook Configuration

- FR-032: A GitHub webhook shall be configured on the repository pointing to the ngrok HTTPS tunnel URL followed by /github-webhook/ (e.g., https://<ngrok_static_domain>.ngrok-free.app/github-webhook/).
- FR-033: The webhook shall be triggered on Push events only. The content type shall be application/json.
- FR-034: The webhook shall use a shared secret (HMAC-SHA256) stored in Vault at secret/github under the key webhook_secret, and configured in the Jenkins GitHub plugin to validate incoming payloads.
- FR-035: On receiving a valid push event to the main branch, the webhook shall trigger the Jenkins pipeline within 30 seconds.

5. CI/CD Pipeline Requirements (Jenkins)

5.1 Jenkins Installation and Configuration

- FR-036: Jenkins LTS shall be installed on the host machine. The recommended method is the Jenkins WAR file or the official apt repository for Ubuntu host systems.
- FR-037: The following Jenkins plugins shall be installed and configured: Git Plugin, GitHub Integration Plugin, Pipeline Plugin (Blue Ocean), Docker Pipeline Plugin, Ansible Plugin, HashiCorp Vault Plugin, NodeJS Plugin (for Newman), and JUnit Plugin.
- FR-038: Jenkins shall be configured with a GitHub credential of type Secret Text storing the GitHub PAT, retrieved from Vault during pipeline execution (not hardcoded in Jenkins credentials store).
- FR-039: The GitHub plugin in Jenkins shall be configured to use the ngrok tunnel URL as the Jenkins URL in Manage Jenkins > Configure System > GitHub > Advanced > Override Hook URL.

5.2 Jenkinsfile — Pipeline Stages

The pipeline shall be defined as a Declarative Pipeline in a Jenkinsfile at the root of the repository. The pipeline shall contain the following stages executed sequentially:

Stage 1: Checkout

- FR-040: Jenkins shall checkout the latest commit from the main branch of the GitHub repository using the Git SCM configuration with the GitHub PAT credential.
- FR-041: The pipeline shall set the environment variable GIT_COMMIT_SHORT to the first 8 characters of the current commit SHA using: env.GIT_COMMIT_SHORT = sh(script: 'git rev-parse --short=8 HEAD', returnStdout: true).trim().

Stage 2: Unit Test

- FR-042: The pipeline shall install Python dependencies into a virtual environment: python3 -m venv venv && venv/bin/pip install -r app/requirements.txt -r tests/requirements-test.txt.
- FR-043: The pipeline shall execute pytest tests/ -v --junitxml=test-results.xml to run all unit tests in the tests/ directory.
- FR-044: The pipeline shall publish the JUnit XML report using the junit 'test-results.xml' post-step so test results appear in the Jenkins UI.
- FR-045: If any test fails (non-zero pytest exit code), the pipeline shall abort immediately at this stage and mark the build as FAILURE. Subsequent stages shall not execute.

Stage 3: Docker Build

- FR-046: The pipeline shall build the Docker image using the Dockerfile at docker/Dockerfile and the build context at the repository root: docker build -t [image]:latest -t [image]:${GIT_COMMIT_SHORT} -f docker/Dockerfile ..

- FR-047: The pipeline shall verify the image was built successfully by running docker inspect [image]:latest and checking the exit code.

Stage 4: Docker Push

- FR-048: The pipeline shall retrieve Docker Hub credentials (username and password) from HashiCorp Vault using the Jenkins Vault plugin. The Vault path shall be secret/dockerhub, and the keys shall be username and password.
- FR-049: The pipeline shall authenticate with Docker Hub using docker login -u $DOCKER_USER -p $DOCKER_PASS and then execute docker push for both tags (latest and commit SHA). The Docker password shall never appear in the Jenkins console log (use withCredentials or Vault plugin secret masking).
- FR-050: After a successful push, the pipeline shall execute docker logout to clear the local credential cache.

Stage 5: Ansible Provision

- FR-051: The pipeline shall invoke the Ansible playbook ansible/site.yml targeting the VM1 inventory group defined in ansible/inventory/hosts.ini.
- FR-052: The SSH private key for the Minikube VM shall be stored in Vault at secret/ssh under the key private_key. The pipeline shall write this key to a temporary file with chmod 600 and pass it to ansible-playbook via --private-key. The file shall be deleted in the pipeline's post { always {} } block.
- FR-053: Ansible shall run with the --diff flag so changes are visible in the Jenkins console log. Ansible shall also run with -e "docker_image=[image]:${GIT_COMMIT_SHORT}" to pass the new image tag to the deployment role.
- FR-054: The Ansible stage shall be idempotent — running it on an already-configured VM shall produce no changes (changed=0).

Stage 6: Kubernetes Deploy

- FR-055: The pipeline shall retrieve the Minikube kubeconfig file content from Vault at secret/kubeconfig under the key config. It shall write this content to a temporary file at /tmp/kubeconfig_${BUILD_NUMBER} and set the environment variable KUBECONFIG to this path.
- FR-056: The pipeline shall update the image tag in the Kubernetes deployment using: kubectl set image deployment/fraudguard-app fraudguard=[image]:${GIT_COMMIT_SHORT} --kubeconfig=$KUBECONFIG.
- FR-057: The pipeline shall wait for the rolling update to complete with a 120-second timeout: kubectl rollout status deployment/fraudguard-app --timeout=120s --kubeconfig=$KUBECONFIG.
- FR-058: If the rollout fails within the timeout, the pipeline shall execute kubectl rollout undo deployment/fraudguard-app --kubeconfig=$KUBECONFIG to roll back to the previous image and mark the build as FAILURE.

Stage 7: Newman Smoke Test

- FR-059: After successful deployment, the pipeline shall wait 15 seconds for the pod to become Ready (sleep 15).
- FR-060: The pipeline shall execute the Newman Postman collection: newman run tests/postman/FraudGuard.postman_collection.json --env-var baseUrl=http://[VM1_IP]:30080 --reporters cli,junit --reporter-junit-export newman-results.xml.
- FR-061: The Newman collection shall contain at least three test cases: (1) POST /predict with a known-fraudulent transaction payload and assert HTTP 200 + prediction==1 + confidence_score > 0.5. (2) POST /predict with a known-legitimate transaction payload and assert HTTP 200 + prediction==0. (3) GET /health and assert HTTP 200 + status=='healthy'.
- FR-062: The pipeline shall publish the Newman JUnit report. If any Newman test fails, the build shall be marked UNSTABLE (not FAILURE, to distinguish deployment issues from regression issues).

5.3 Pipeline Post Actions

- FR-063: The pipeline post { always {} } block shall: delete the temporary kubeconfig file, delete the temporary SSH key file, and send a build status notification to a Slack channel or email (optional but recommended for innovation marks).
- FR-064: The pipeline post { success {} } block shall print: 'Deployment of [image]:${GIT_COMMIT_SHORT} completed successfully. Application available at http://[VM1_IP]:30080'.
- FR-065: The total pipeline execution time from Checkout to Newman smoke test completion shall not exceed 15 minutes on the target hardware.

6. Configuration Management Requirements (Ansible)

6.1 Inventory

- FR-066: The Ansible inventory file ansible/inventory/hosts.ini shall define a group named [minikube_vm] containing the VM1 SSH details: VM IP address (or localhost port forward), SSH user (ubuntu), SSH port, and reference to the private key variable (ansible_ssh_private_key_file={{ ssh_key_path }}).
- FR-067: Group variables shall be stored in ansible/inventory/group_vars/minikube_vm.yml and shall include: ansible_python_interpreter=/usr/bin/python3, minikube_memory=9g, minikube_cpus=4, and docker_compose_version=2.24.6.

6.2 Role: common

- FR-068: The common role shall execute the following tasks in order: apt-get update, apt-get upgrade (with DEBIAN_FRONTEND=noninteractive), install packages: curl, wget, git, vim, net-tools, ca-certificates, gnupg, lsb-release, python3-pip, python3-venv.
- FR-069: The common role shall configure the system timezone to UTC using the timezone Ansible module.
- FR-070: The common role shall disable the UFW firewall (for development simplicity) using the ufw module with state: disabled.

6.3 Role: docker

- FR-071: The docker role shall add the official Docker apt repository GPG key and repository source to the VM, then install docker-ce, docker-ce-cli, containerd.io, docker-buildx-plugin, and docker-compose-plugin.
- FR-072: The docker role shall ensure the Docker service is started and enabled at boot using the service module.
- FR-073: The docker role shall add the ubuntu user to the docker group using the user module, enabling Docker commands without sudo.
- FR-074: The docker role shall be fully idempotent — re-running on an already-configured VM shall report changed=0 for all tasks.

6.4 Role: kubernetes

- FR-075: The kubernetes role shall install kubectl version 1.29 by downloading the binary from the official Kubernetes release URL and placing it at /usr/local/bin/kubectl with chmod 755.
- FR-076: The kubernetes role shall install Minikube version 1.32 by downloading the .deb package from the official GitHub releases page and installing it with dpkg.
- FR-077: The kubernetes role shall start Minikube with the Docker driver using: minikube start --driver=docker --memory={{ minikube_memory }} --cpus={{ minikube_cpus }} --kubernetes-version=v1.29.0. The become: no flag shall be used since Minikube runs as the ubuntu user.
- FR-078: The kubernetes role shall enable the metrics-server addon: minikube addons enable metrics-server, which is required for HPA CPU-based autoscaling.

- FR-079: The kubernetes role shall verify that the Minikube node is Ready by running kubectl get nodes and checking the output with assert_that.

6.5 Master Playbook (site.yml)

- FR-080: The master playbook ansible/site.yml shall import the three roles in order: common, docker, kubernetes. It shall also include a final task that applies all K8s manifests from the k8s/ directory using kubectl apply -f k8s/.
- FR-081: The playbook shall include a handlers section defining a Restart docker handler that executes systemctl restart docker and is notified by the docker role when the Docker service configuration changes.

7. Orchestration and Scaling Requirements (Kubernetes)

7.1 Application Deployment

- FR-082: The application Deployment manifest (k8s/deployment.yaml) shall define a Deployment named fraudguard-app in the namespace fraudguard (namespace must be created before apply).
- FR-083: The Deployment shall specify: replicas: 2 as the initial replica count, the container image from Docker Hub, containerPort: 8000, and resource requests and limits: requests: {cpu: '250m', memory: '256Mi'}, limits: {cpu: '500m', memory: '512Mi'}.
- FR-084: The Deployment strategy shall be RollingUpdate with maxSurge: 1 and maxUnavailable: 0, ensuring zero downtime during image updates.
- FR-085: The Deployment shall define liveness and readiness probes: livenessProbe using httpGet on /health at port 8000 with initialDelaySeconds: 15, periodSeconds: 20, failureThreshold: 3. readinessProbe using httpGet on /health with initialDelaySeconds: 10, periodSeconds: 10.
- FR-086: The pod spec shall define two containers in the same pod: the main fraudguard container and a filebeat sidecar container. Both shall mount the volume named app-logs at /var/log/app using an emptyDir volume.

7.1.1 Filebeat Sidecar Specification

- FR-087: The Filebeat sidecar shall use the image docker.elastic.co/beats/filebeat:8.13.0.
- FR-088: The Filebeat sidecar shall mount the shared emptyDir volume at /var/log/app (read-only) and a ConfigMap volume containing the filebeat.yml configuration at /usr/share/filebeat/filebeat.yml.
- FR-089: The filebeat.yml ConfigMap shall configure: filebeat.inputs of type log pointing to /var/log/app/fraud.log, and output.logstash pointing to logstash-service:5044 (the Logstash ClusterIP service within the cluster). The multiline configuration shall be disabled since logs are single-line JSON.

7.2 Application Service

- FR-090: The Service manifest (k8s/service.yaml) shall create a NodePort Service named fraudguard-service selecting pods with label app: fraudguard-app. The service shall expose port 80 targeting container port 8000 with nodePort: 30080.
- FR-091: The application shall be accessible from the host machine at http://[VM1_IP]:30080 after port forwarding is configured on the VM.

7.3 Horizontal Pod Autoscaler (HPA)

- FR-092: An HPA resource shall be created targeting the fraudguard-app Deployment with minReplicas: 2 and maxReplicas: 6.
- FR-093: The HPA scaling metric shall be CPU utilisation with a target average utilisation of 70%. The HPA shall use the autoscaling/v2 API version.

- FR-094: The HPA stabilisationWindow shall be set to 60 seconds for scale-down to prevent rapid pod termination after a load test: behavior.scaleDown.stabilizationWindowSeconds: 60.
- FR-095: The HPA demonstration procedure during evaluation shall use Locust (pip install locust) to generate load: locust -f locustfile.py --headless -u 50 -r 10 --host http://[VM1_IP]:30080 --run-time 120s. The evaluator shall observe pods scaling from 2 to a higher count using kubectl get hpa -w.

7.4 ELK Stack Kubernetes Deployments

7.4.1 Elasticsearch

- FR-096: Elasticsearch shall be deployed as a Deployment named elasticsearch in the fraudguard namespace using image elasticsearch:8.13.0.
- FR-097: The Elasticsearch container shall set the environment variables: discovery.type=single-node, ES_JAVA_OPTS=-Xms512m -Xmx512m (critical to prevent OOMKill inside Minikube), xpack.security.enabled=false (disabled for academic simplicity).
- FR-098: A ClusterIP Service named elasticsearch-service shall expose port 9200 internally within the cluster.

7.4.2 Logstash

- FR-099: Logstash shall be deployed as a Deployment named logstash using image logstash:8.13.0.
- FR-100: Logstash shall be configured via a ConfigMap containing a pipeline.conf file with: input { beats { port => 5044 } }, filter { json { source => 'message' } mutate { add_field => { 'app' => 'fraudguard' } } }, and output { elasticsearch { hosts => ['elasticsearch-service:9200'] index => 'fraudguard-logs-%{+YYYY.MM.dd}' } }.
- FR-101: A ClusterIP Service named logstash-service shall expose port 5044 (Beats input) internally. Filebeat sidecar containers shall ship logs to logstash-service:5044.

7.4.3 Kibana

- FR-102: Kibana shall be deployed as a Deployment named kibana using image kibana:8.13.0 with the environment variable ELASTICSEARCH_HOSTS=http://elasticsearch-service:9200.
- FR-103: A NodePort Service named kibana-service shall expose Kibana's port 5601 with nodePort: 30601, making the Kibana UI accessible at http://[VM1_IP]:30601 from the host.
- FR-104: Upon first launch, the Kibana index pattern fraudguard-logs-* shall be configured, and a dashboard named FraudGuard Monitoring Dashboard shall be created containing the following visualisations: (1) Fraud Rate Over Time — bar chart of prediction==1 vs prediction==0 by timestamp (per-minute interval). (2) Confidence Score Distribution — histogram of confidence_score values across all predictions. (3) Flagged Transactions Table — data table showing the most recent 50 records where prediction==1, with columns transaction_id, confidence_score, and timestamp.

8. Secret Management Requirements (HashiCorp Vault)

8.1 Vault Installation and Initialisation

- FR-105: HashiCorp Vault shall be installed on the host machine using the official HashiCorp apt repository (Ubuntu host) or downloaded as a binary.
- FR-106: For this academic project, Vault shall be started in development mode: vault server -dev -dev-root-token-id=root. The root token root shall be used only during initial setup. All pipeline access shall use AppRole authentication.
- FR-107: The KV secrets engine version 2 shall be enabled at the path secret/: vault secrets enable -path=secret kv-v2.

8.2 Secret Paths

Vault Path	Key(s)	Description
secret/dockerhub	username, password	Docker Hub account credentials for image push
secret/github	pat, webhook_secret	GitHub PAT for repo access; HMAC secret for webhook validation
secret/kubeconfig	config	Full content of ~/.kube/config from Minikube VM, base64-encoded
secret/ssh	private_key	SSH private key for Ansible to access the Minikube VM

8.3 AppRole Authentication

- FR-108: AppRole auth shall be enabled: vault auth enable approle. A Vault policy named jenkins-policy shall be created granting read access to all paths under secret/data/: path 'secret/data/*' { capabilities = ['read'] }.
- FR-109: An AppRole role named jenkins-role shall be created with the jenkins-policy attached: vault write auth/approle/role/jenkins-role token_policies='jenkins-policy' token_ttl=1h token_max_ttl=4h.
- FR-110: The Role ID and Secret ID shall be retrieved and stored in the Jenkins credentials store as two separate Secret Text credentials with IDs VAULT_ROLE_ID and VAULT_SECRET_ID respectively. These are the only credentials stored in Jenkins; all other secrets live in Vault.
- FR-111: The Jenkins HashiCorp Vault Plugin shall be configured with: Vault URL http://localhost:8200, Authentication type AppRole, Role ID and Secret ID credentials as defined above, and engine version KV2.

8.4 Vault Usage in Jenkinsfile

- FR-112: The Jenkinsfile shall use the withVault block to retrieve secrets: withVault(configuration: [vaultUrl: 'http://localhost:8200', vaultCredentialId: 'vault-approle'], vaultSecrets: [[path: 'secret/dockerhub', secretValues: [[envVar: 'DOCKER_USER', vaultKey: 'username'], [envVar: 'DOCKER_PASS', vaultKey: 'password']]]]) { ... }

- FR-113: The DOCKER_PASS environment variable retrieved from Vault shall be automatically masked in the Jenkins console log by the Vault plugin (replaced with ****).
- FR-114: No secret value shall appear in the Jenkinsfile, in any Ansible playbook, in any K8s manifest, or in the Git repository at any point.

9. Testing Requirements

9.1 Unit Tests (pytest)

- FR-115: The test suite at tests/test_predict.py shall contain a minimum of eight test cases covering: model loading (test_model_loads_successfully), prediction output type (test_prediction_returns_int), confidence score range (test_confidence_score_between_0_and_1), /predict returns 200 for valid payload (test_predict_valid_payload), /predict returns 422 for missing fields (test_predict_missing_field), /health returns 200 and correct JSON (test_health_endpoint), /predict returns label 'fraudulent' for prediction==1 (test_label_fraudulent), /predict returns label 'legitimate' for prediction==0 (test_label_legitimate).
- FR-116: Tests shall use FastAPI's TestClient from starlette.testclient for endpoint testing, avoiding the need for a running server during CI.
- FR-117: Fixture: a conftest.py file shall define a mock_transaction_fraud fixture returning a dict with all required TransactionRequest fields, pre-set to values known to produce a fraudulent prediction from the trained model.
- FR-118: The tests/ directory shall include a requirements-test.txt with: pytest>=8.1.0, pytest-cov>=5.0.0, httpx>=0.27.0 (required by TestClient).
- FR-119: The pytest configuration (pytest.ini or pyproject.toml) shall enable coverage reporting: addopts = --cov=app --cov-report=term-missing --cov-fail-under=70. Builds shall fail if coverage drops below 70%.

9.2 Smoke Tests (Newman / Postman)

- FR-120: The Postman collection shall define an environment variable baseUrl that is set at runtime by Newman. No hardcoded IP addresses shall appear inside the collection JSON.
- FR-121: Test Case 1 — Fraud Detection: POST {{baseUrl}}/predict with a JSON body containing all required features. Assertions: response status 200, response body contains field prediction of type number, response body prediction value equals 1, response body confidence_score is greater than 0.5.
- FR-122: Test Case 2 — Legitimate Transaction: POST {{baseUrl}}/predict with a different payload known to produce prediction==0. Assertions: status 200, prediction equals 0.
- FR-123: Test Case 3 — Health Check: GET {{baseUrl}}/health. Assertions: status 200, body.status equals 'healthy', body.model_loaded equals true.
- FR-124: Test Case 4 — Invalid Payload: POST {{baseUrl}}/predict with an empty JSON body {}. Assertion: status 422.
- FR-125: Node.js and Newman shall be installed on the host machine (the Jenkins server). Newman version 6.x shall be used: npm install -g newman newman-reporter-junitfull.

10. Non-Functional Requirements

10.1 Performance

- NFR-001: The /predict endpoint shall respond within 500 milliseconds (p95 latency) for a single prediction request under normal load (1 concurrent user).
- NFR-002: The application shall sustain a throughput of at least 20 requests per second before CPU utilisation exceeds 70% on a 2-pod deployment (triggering HPA), as demonstrated by Locust.
- NFR-003: The Jenkins pipeline (from webhook trigger to successful Newman smoke test) shall complete in under 15 minutes on the target hardware (16 GB RAM host, 4-core CPU allocated to Minikube VM).
- NFR-004: The Kibana dashboard shall load and display the most recent logs within 30 seconds of a prediction request being made.

10.2 Security

- NFR-005: No credential (password, PAT, API key, SSH private key) shall be stored in the Git repository, Jenkinsfile, Ansible playbook, or Kubernetes manifest in plaintext. All secrets shall be stored exclusively in HashiCorp Vault.
- NFR-006: The Docker container shall run as a non-root user (appuser, UID 1000). The Dockerfile shall not use USER root after the build stage.
- NFR-007: The Vault AppRole Secret ID shall have a TTL of no more than 4 hours, limiting the blast radius of a credential leak.
- NFR-008: The GitHub webhook shall use HMAC-SHA256 signature validation. Jenkins shall reject any webhook payload that does not pass signature verification.
- NFR-009: Docker images shall not contain pip-installed packages from untrusted sources. All packages shall be pinned to specific versions in requirements.txt.

10.3 Availability

- NFR-010: The Kubernetes Deployment shall maintain at least 2 running pods at all times during normal operation. The HPA minReplicas: 2 setting enforces this.
- NFR-011: The rolling update strategy (maxUnavailable: 0) shall ensure zero application downtime during a new image deployment. The application shall be continuously available (returning HTTP 200 on /health) throughout the deployment process.
- NFR-012: If the application pod crashes (liveness probe fails 3 consecutive times), Kubernetes shall automatically restart it without manual intervention.

10.4 Scalability

- NFR-013: The application shall scale from 2 to 6 pods automatically when CPU utilisation exceeds 70% for 60 consecutive seconds, as managed by the HPA.
- NFR-014: The system architecture shall support adding a second Minikube node (multi-node cluster) without changes to the application code, Dockerfile, or K8s manifests. Only the Minikube start command would change.

10.5 Maintainability and Reproducibility

- NFR-015: Every infrastructure component (VM configuration, K8s manifests, Ansible roles) shall be defined as code in the Git repository. A fresh deployment from a clean VM shall be reproducible by running: (1) ansible-playbook ansible/site.yml, (2) kubectl apply -f k8s/, with no additional manual steps beyond initial Vault secret seeding.
- NFR-016: The Ansible playbooks shall be fully idempotent. Re-running any playbook on an already-configured system shall produce no unintended side effects (changed=0 for all tasks).
- NFR-017: All configuration values (Minikube memory, Docker image name, Elasticsearch heap size, NodePort numbers) shall be parameterised as variables in Ansible group_vars or Kubernetes ConfigMaps. No magic numbers shall appear inline in task files or manifests.

10.6 Observability

- NFR-018: 100% of prediction requests shall produce a structured JSON log entry. No prediction shall be silently processed without a corresponding log record in fraud.log.
- NFR-019: The Kibana FraudGuard Monitoring Dashboard shall be the primary evaluation artefact for the ELK requirement. The dashboard shall be exportable as a JSON file (for reproducibility) and committed to the repository at kibana/dashboard_export.ndjson.
- NFR-020: Log records shall appear in Kibana within 30 seconds of generation (accounting for Filebeat shipping interval, Logstash processing, and Elasticsearch indexing latency).

11. Evaluation Marks Alignment

The following table maps each evaluation criterion to the specific requirements in this document and to the demonstration evidence expected during evaluation.

Criterion	Marks	Requirement Coverage	Evaluation Evidence
Working CI/CD Pipeline	20	FR-032–FR-065, FR-074, FR-080	Live demo: push a trivial code change (e.g., update a comment); evaluator watches Jenkins pipeline run all 7 stages to completion and confirms new change visible in browser at port 30080
ELK Logs in Kibana	Included in 20	FR-086–FR-089, FR-096–FR-104, NFR-018–NFR-020	Evaluator sends a POST to /predict; within 30 seconds the Kibana dashboard shows the new log entry with fraud classification details
Ansible Roles	1 of 3	FR-066–FR-081	Show site.yml importing 3 roles; show roles/ directory structure; run ansible-playbook --check and demonstrate changed=0
HashiCorp Vault	1 of 3	FR-105–FR-114	Show Vault UI with secret paths; show Jenkins pipeline log with masked DOCKER_PASS; show that Jenkinsfile contains no hardcoded credentials
Kubernetes HPA	1 of 3	FR-092–FR-095, NFR-013	Run locust load test; show kubectl get hpa -w output scaling pods from 2 to higher count; show pods scaling back down after load ends
Domain Innovation	2	FR-001–FR-018, FR-104, NFR-018	Kibana dashboard showing fraud-specific metrics; FastAPI /docs showing ML prediction endpoint; domain justification in README

12. Implementation Build Order and Risk Register

12.1 Recommended Build Order

Given the solo nature of the project and a 2–4 week timeline, the following sequential build order minimises compounding failures — each phase must be working before the next begins.

Phase	Duration	Tasks	Exit Criterion
1	Days 1–2	Train model, build FastAPI app, write pytest tests, write Dockerfile, run locally with docker run	curl localhost:8000/predict returns a valid JSON fraud prediction
2	Day 3	Push to GitHub, configure ngrok, set up Jenkins, create Jenkinsfile with stages 1–3 (Checkout, Test, Build)	Jenkins pipeline turns green on stages 1–3 when code is pushed
3	Day 4	Seed Vault secrets, configure Jenkins Vault plugin, add Docker Push stage	Jenkins pushes image to Docker Hub successfully; DOCKER_PASS is masked in logs
4	Days 5–6	Create and configure VM1, write Ansible roles, add Ansible stage to pipeline	ansible-playbook runs successfully; VM has Docker, kubectl, Minikube running
5	Days 7–8	Write K8s manifests (Deployment, Service, HPA), add kubectl deploy stage, write Newman collection	kubectl rollout status shows successful deployment; Newman smoke tests pass
6	Days 9–11	Deploy ELK to K8s, configure Filebeat ConfigMap, verify logs in Kibana, build Kibana dashboard	Kibana shows fraud logs within 30 seconds of prediction request
7	Days 12–13	HPA stress test with Locust; verify scaling; full end-to-end demo rehearsal	Full pipeline runs in under 15 min; HPA scales pods during Locust test
8	Days 14+	Write README, export Kibana dashboard JSON, clean up code, rehearse evaluation demo	Project is fully reproducible from a clean state following README instructions

12.2 Risk Register

Risk	Likelihood	Impact	Mitigation
ngrok session expires, breaking GitHub webhook	High	High	Use ngrok static domain (free tier): ngrok config add-authtoken ... then ngrok http --domain=<fixed>.ngrok-free.app 8080. Commit the domain to README so it can be restored instantly

Risk	Likelihood	Impact	Mitigation
Elasticsearch OOMKilled inside Minikube	High	High	Set ES_JAVA_OPTS=-Xms512m -Xmx512m in the Elasticsearch Deployment. Monitor with kubectl top pods -n fraudguard. Reduce Logstash heap with LS_JAVA_OPTS=-Xms256m -Xmx256m
model.pkl incompatibility between train and serve environments	Medium	High	Pin scikit-learn==1.4.2 in both train requirements and app requirements.txt. Train in a Docker container using the same image as the production container
Vault token expires mid-pipeline	Medium	Medium	Set token_ttl=1h and token_max_ttl=4h on the AppRole. Jenkins pipelines run in under 15 minutes, well within 1 hour TTL
Minikube IP changes after VM reboot	Medium	Medium	Use minikube ip command in the Jenkinsfile and Newman --env-var to dynamically resolve the IP rather than hardcoding it
Ansible idempotency failures on re-run	Low	Medium	Test with --check --diff before adding to pipeline. Use creates: parameter on shell tasks to prevent re-running completed steps
HPA not scaling (metrics-server not active)	Medium	Medium	Run minikube addons enable metrics-server and verify with kubectl top nodes before the evaluation demo. Include this as a post-provision check in the Ansible kubernetes role
Filebeat logs not appearing in Kibana	Medium	Medium	Debug pipeline: exec into Filebeat sidecar and check filebeat logs. Verify logstash-service DNS resolves inside cluster with kubectl exec. Verify index pattern created in Kibana

Appendix A: Tool and Version Reference

Tool / Component	Version	Role in System
Python	3.11.x	Application runtime and model training
FastAPI	>=0.110.0	Web framework for prediction API
scikit-learn	1.4.2 (pinned)	Random Forest model training and inference
Docker Engine	24.x	Container build and runtime
Jenkins LTS	2.452.x	CI/CD pipeline orchestration
Ansible	9.x (core 2.16)	Infrastructure provisioning (IaC)
Minikube	1.32.x	Local Kubernetes cluster
kubectl	1.29.x	Kubernetes CLI for deployments
HashiCorp Vault	1.16.x	Centralised secret management
ngrok Agent	3.x	Localhost tunnelling for GitHub webhook
Elasticsearch	8.13.0	Log storage and search index
Logstash	8.13.0	Log ingestion pipeline
Kibana	8.13.0	Log visualisation dashboard
Filebeat	8.13.0	Log shipping sidecar
Node.js	20 LTS	Newman runtime
Newman	6.x	Postman collection CLI runner for smoke tests
Locust	2.x	Load testing tool for HPA demonstration
VirtualBox	7.x	VM hypervisor for Minikube host
Ubuntu Server	22.04 LTS	Guest OS for Minikube VM
Git	2.43.x	Version control
Git LFS	3.5.x	Large file storage for model.pkl

Appendix B: Requirement ID Index

Functional Requirements: FR-001 through FR-125 (125 total requirements across all system components).

Non-Functional Requirements: NFR-001 through NFR-020 (20 total requirements across performance, security, availability, scalability, maintainability, and observability).

Total Requirements: 145.

Document End.